

IF2211 Strategi Algoritma

Tugas Kecil 3: Ice Sliding Puzzle Solver



Disusun oleh

Ni Made Sekar Jelita (18223101)

Kelas K1

FAKULTAS SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi.....	2
Daftar Tabel.....	3
Penjelasan Penerapan Pathfinding: UCS, GBFS, dan A*	4
A. Algoritma Pathfinding yang Digunakan.....	4
1. Algoritma A*	4
2. Algoritma Uniform Cost Search (UCS)	4
3. Algoritma Greedy Best-First Search (GBFS)	5
B. Implementasi Algoritma Pathfinding.....	5
1. Definisi $f(n)$ dan $g(n)$ pada algoritma.....	5
2. Admissibility Heuristik A*	5
3. Perbandingan algoritma yang diimplementasikan.....	5
Hasil Pengujian.....	7
LAMPIRAN.....	15
Tabel Checklist.....	15
Pranala.....	15
Pernyataan.....	16

Daftar Tabel

Tabel 1 Hasil Uji tc_soal.txt.....	7
Tabel 2 Hasil Uji tc1.txt.....	8
Tabel 3 Hasil Uji tc2.txt.....	10
Tabel 4 Hasil Uji tc3.txt.....	11
Tabel 5 Hasil Uji tc4.txt.....	13

Penjelasan Penerapan Pathfinding: UCS, GBFS, dan A*

A. Algoritma Pathfinding yang Digunakan

1. Algoritma A*

A* adalah algoritma pencarian jalur yang menggabungkan jarak tempuh dan estimasi heuristik untuk mencapai rute terpendek secara optimal dan efisien. Algoritma ini sangat populer karena mampu menemukan solusi terbaik dalam graf berbobot dengan menyeimbangkan biaya akumulatif dan jarak ke target.

Berikut adalah langkah-langkah dari algoritma A*.

- Tentukan target saat ini, dapat diambil dari angka terkecil yang belum dikunjungi atau Target
- Masukkan Node awal ke Priority Queue dengan nilai prioritas $f(n) = g(n) + h(n)$. Di sini, $g(n)$ adalah akumulasi *cost* dan $h(n)$ adalah *Manhattan Distance*.
- Ambil Node dengan nilai $f(n)$ terkecil dari Priority Queue untuk dieksplorasi
- Eksplor tetangga dengan mekanisme *sliding* dan perbarui nilai $g(n)$ berdasarkan total *cost* petak yang dilewati
- Simpan Node baru ke Priority Queue jika ditemukan jalur yang lebih murah. Ulangi proses tersebut hingga target tercapai secara optimal

2. Algoritma Uniform Cost Search (UCS)

Uniform Cost Search adalah varian dari algoritma Dijkstra yang fokus mencari jalur dengan total *cost* terendah dengan cara mengeksplorasi simpul berdasarkan akumulasi bobot terkecil dari titik awal. Algoritma ini sangat pandai menemukan solusi paling optimal pada graf yang memiliki *weight* berbeda-beda. Meskipun begitu, proses pencariannya cenderung lebih luas dan memakan waktu.

Berikut adalah langkah-langkah dari algoritma UCS.

- Masukkan Node awal ke dalam Priority Queue dengan *cost* 0
- Ambil Node dengan *cost* akumulasi terendah dari PQ
- Lakukan eksplorasi tetangga dengan mekanisme *sliding*
- Hitung total *cost* baru. Jika Node baru belum dikunjungi atau ditemukan jalur lebih murah, masukkan ke PQ
- Ulangi hingga target angka atau target akhir (0) tercapai

3. Algoritma Greedy Best-First Search (GBFS)

Greedy Best-First Search adalah algoritma AI yang memprioritaskan jalur yang tampak paling menjanjikan berdasarkan estimasi jarak ke target tanpa mempedulikan *cost* di tiap *step* yang sudah ditempuh. Meskipun sangat cepat karena fokus langsung pada tujuan, algoritma ini tidak menjamin penemuan jalur terpendek dan bisa terjebak pada solusi sub-optimal.

Berikut adalah langkah-langkah dari algoritma GBFS.

- Tentukan target saat ini, dapat diambil dari angka terkecil yang belum dikunjungi atau Target
- Masukkan Node awal ke PriorityQueue dengan nilai prioritas berdasarkan Heuristic
- Ambil Node yang paling dekat dengan Target secara geografi
- Eksplor tetangga tanpa mempedulikan *cost* tiap *step* ($g(n)$)
- Dapat terjebak di *local optima*

B. Implementasi Algoritma Pathfinding

1. Definisi $f(n)$ dan $g(n)$ pada algoritma

Pada algoritma yang diterapkan, $f(n)$ didefinisikan sebagai xx , sedangkan $g(n)$ sebagai ...

- $g(n)$: Akumulasi nilai *cost* dari setiap petak yang dilewati selama meluncur
- $f(n)$: Total estimasi biaya jalur termurah yang melalui simpul n . Pada A^* , nilainya adalah $g(n) + h(n)$

2. Admissibility Heuristik A^*

Pada tugas ini, heuristik yang digunakan adalah Manhattan Distance. Heuristik ini bersifat *admissible* karena menghitung jarak garis lurus antarkoordinat. Selain itu, estimasi $h(n)$ juga tidak akan melebihi *cost* sebenarnya ($h^*(n)$) karena tidak ada jalur yang lebih pendek selain jalur garis lurus di dalam sebuah *grid*. Oleh karena itu, A^* dijamin akan selalu menemukan solusi optimal.

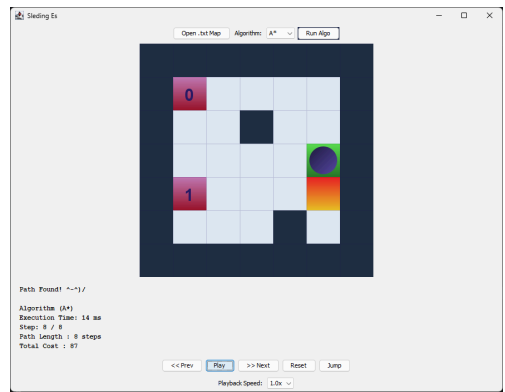
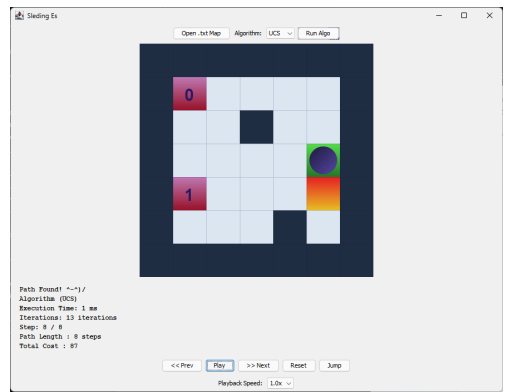
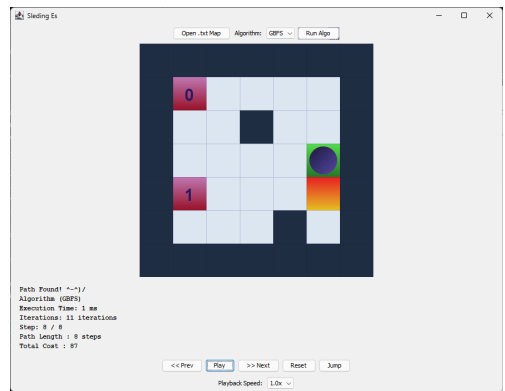
3. Perbandingan algoritma yang diimplementasikan

Secara teori, UCS merupakan algoritma *uninformed search* yang optimal dan berfokus sepenuhnya pada akumulasi *cost* ($g(n)$), namun proses pencariannya cenderung lambat karena mengeksplorasi semua arah tanpa panduan arah target. Sebaliknya, GBFS adalah algoritma *informed search* yang sangat cepat karena hanya memprioritaskan estimasi jarak heuristik ke target ($h(n)$), tetapi algoritma ini tidak menjamin solusi optimal

dan berisiko gagal menemukan jalur jika terjebak dalam rintangan atau *loop*. Sementara itu, algoritma A* menggabungkan kelebihan keduanya dengan menggunakan fungsi evaluasi $f(n) = g(n) + h(n)$. Dengan fungsi tersebut, A* mampu menghasilkan solusi yang terjamin optimal dengan arah pencarian yang jauh lebih cerdas dan terfokus dibandingkan UCS maupun GBFS.

Hasil Pengujian

Tabel 1 Hasil Uji tc_soal.txt

Algoritma	Execution Time (ms)	Iterations, Path Length	Total cost	Screenshot
A*	14	13 iterations, 8 steps	87	
UCS	1	13 iterations, 8 steps	87	
GBFS	1	11 iterations, 8 steps	87	

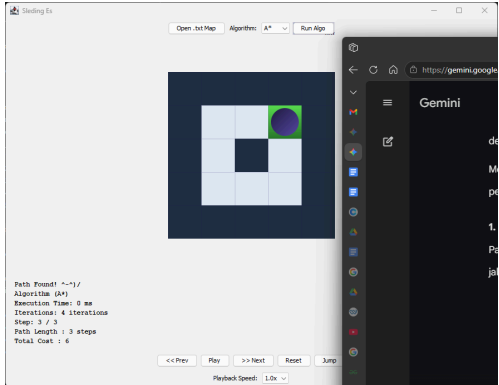
GBFS menjadi yang paling efektif pada *test case* tersebut karena sifatnya yang hanya fokus pada jarak estimasi terdekat tanpa memperdulikan *cost*. Hal ini terjadi karena struktur Map cenderung terbuka tanpa jebakan rintangan sehingga strategi *greedy*-nya langsung menemukan jalur yang kebetulan juga merupakan

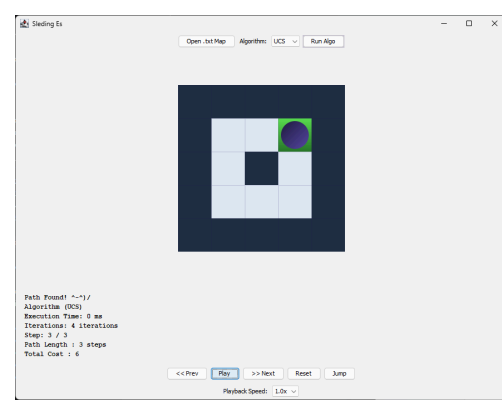
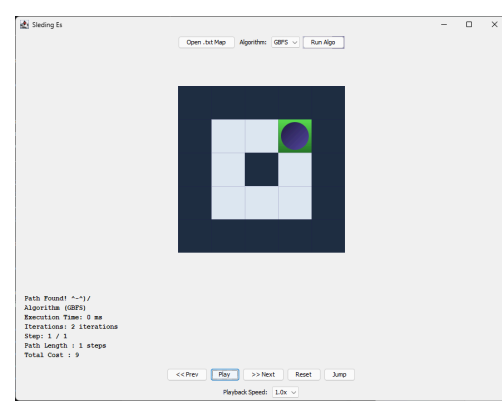
jalur paling optimal. Pada kondisi ini, GBFS berhasil mencapai target lebih cepat karena tidak perlu melakukan validasi *cost* secara keseluruhan seperti yang dilakukan oleh algoritma A* dan UCS.

File tc_soal.txt

tc_soal.txt
77 XXXXXXX X0****X X**X**X X****OX X1***LX XZ**X*X XXXXXXX 999 999 999 999 999 999 999 999 3 5 2 8 1 999 999 7 4 999 6 9 999 999 2 8 3 5 4 999 999 6 1 7 2 999 999 999 9 3 4 999 8 999 999 999 999 999 999 999 999

Tabel 2 Hasil Uji tc1.txt

Algoritma	Execution Time (ms)	Iterations, Path Length	Total cost	Screenshot
A*	0	4 iterations, 3 steps	6	

UCS	0	4 iterations, 3 steps	6	
GBFS	0	2 iterations, 1 steps	9	

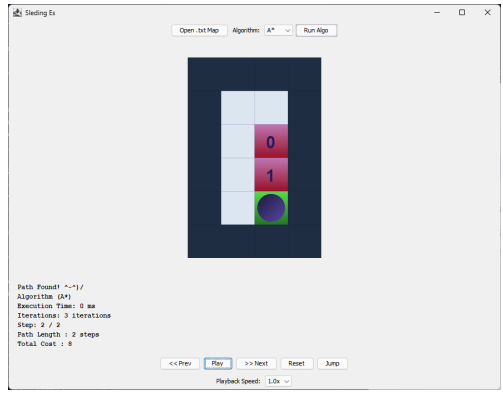
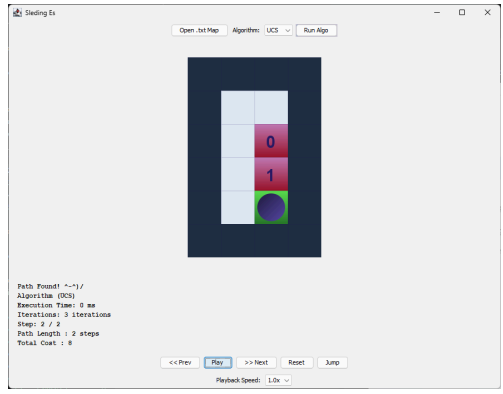
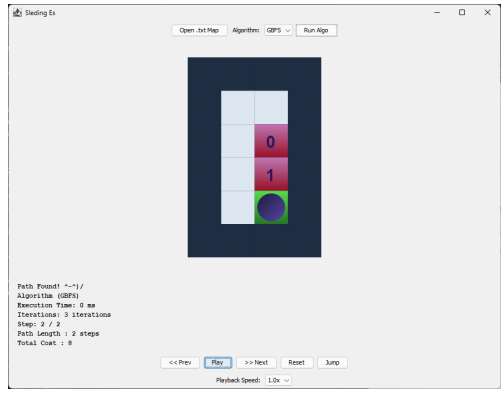
Pada *test case* ini, GBFS terlihat lebih "efektif" secara iterasi karena hanya membutuhkan dua iterasi untuk mencapai target. Akan tetapi, hasilnya tidak optimal karena menghasilkan *total cost* yang jauh lebih tinggi (9) dibandingkan A* dan UCS (6). Hal ini terjadi karena GBFS langsung menuju ke arah target melalui jalur dengan *cost* mahal (8) demi meminimalkan jarak. Sementara itu, A* dan UCS rela mengambil langkah lebih banyak (3 *steps*) demi mendapatkan jalur dengan akumulasi *cost* paling rendah. Perbedaan ini menegaskan bahwa pada peta dengan jebakan *cost*, GBFS mengorbankan kualitas solusi demi kecepatan pencarian, sedangkan A* tetap menjamin hasil termurah.

File tc1.txt

tc_soal.txt
<pre> 5 5 XXXXX XZ*OX X*X*X X***X XXXXX 999 999 999 999 999 999 1 8 1 999 999 1 999 1 999 </pre>

999 1 1 1 999
999 999 999 999 999

Tabel 3 Hasil Uji tc2.txt

Algoritma	Execution Time (ms)	Iterations, Path Length	Total cost	Screenshot
A*	0	3 iterations, 2 steps	8	
UCS	0	3 iterations, 2 steps	8	
GBFS	0	3 iterations, 2 steps	8	

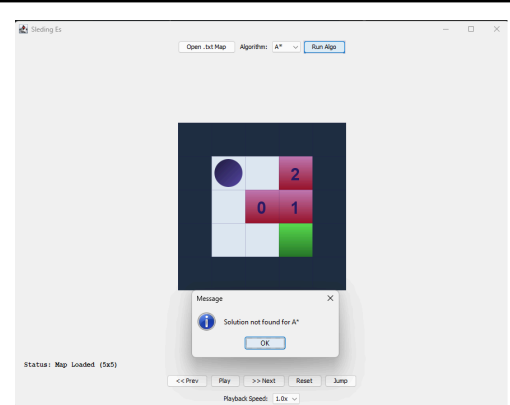
Pada pengujian peta ini, algoritma A* dan UCS akan bekerja lebih keras untuk mengevaluasi setiap petak demi menemukan jalur dengan akumulasi cost paling efisien di tengah sebaran angka yang dinamis. Sebaliknya, GBFS cenderung

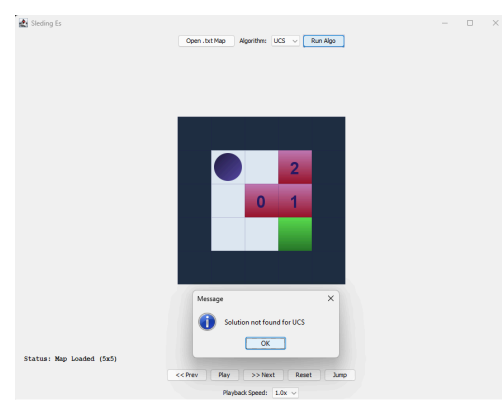
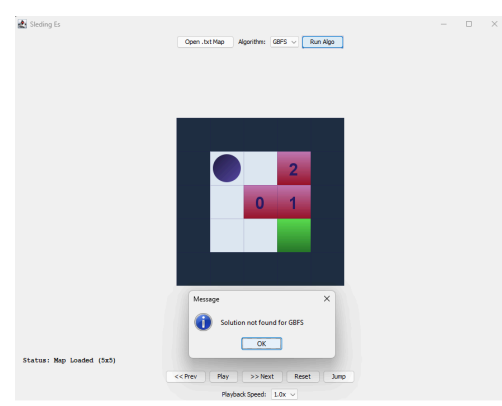
mengabaikan variasi *cost* yang ada dan langsung menuju ke arah target terdekat berdasarkan estimasi jarak Manhattan yang pada peta tipe spiral seperti ini berisiko menghasilkan jalur yang tidak optimal. Perbedaan performa antara ketiga algoritma akan terlihat jelas pada jumlah iterasi dan *total cost* akhir. Algoritma A* tetap menjadi solusi paling seimbang dalam mengoptimalkan pencarian sekaligus menekan biaya perjalanan.

File tc2.txt

tc_soal.txt
<pre> 6 4 XXXX XZ*X X*OX X*1X X*OX XXXX 999 999 999 999 999 1 5 999 999 2 1 999 999 4 1 999 999 3 1 999 999 999 999 999 </pre>

Tabel 4 Hasil Uji tc3.txt

Algoritma	Execution Time (ms)	Iterations, Path Length	Total cost	Screenshot
A*	-	-	-	

UCS	-	-	-	
GBFS	-	-	-	

Map `tc3.txt` ini memaksa algoritma untuk menelusuri angka secara berurutan. Algoritma A* dan UCS akan mengeksplorasi jalur dengan teliti untuk meminimalkan akumulasi *cost* yang meningkat di setiap lapisan spiral. Sementara itu, GBFS akan bergerak lebih agresif menuju target terdekat berdasarkan jarak lurus, tetapi GBFS berisiko terjebak pada jalur dengan *cost* tinggi karena sifatnya yang mengabaikan nilai ekonomi pada setiap petak. Hasil pengujian ini akan menunjukkan keunggulan A* dalam menjaga efisiensi *cost* tanpa harus menggeledah seluruh ruang pencarian seperti yang dilakukan oleh UCS.

Meskipun begitu, hasil pengujian menunjukkan ketiga algoritma gagal menemukan *path* yang tepat. Kegagalan ini disebabkan oleh mekanik seluncur es yang mengharuskan *player* hanya bisa berhenti jika menabrak tembok. Sementara itu, posisi target angka pada `tc3.txt` diletakkan di area terbuka tanpa "penahan" di titik pemberhentian yang tepat.

File `tc3.txt`

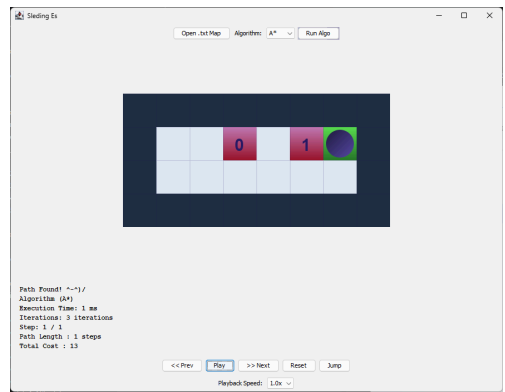
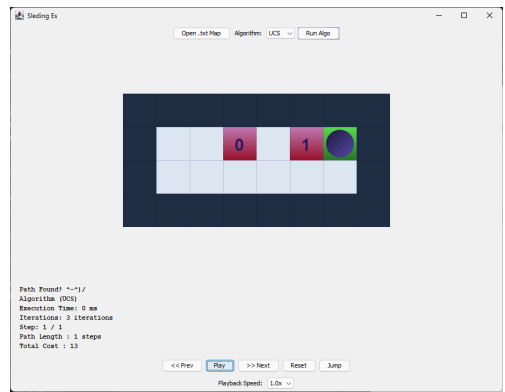
tc_soal.txt
5 5 XXXXX XZ*2X X*01X

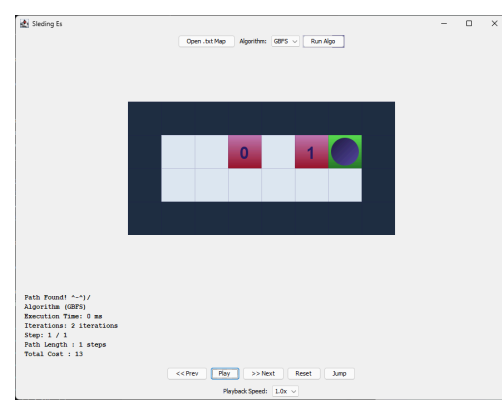
```

X**OX
XXXXX
999 999 999 999 999
999 1 2 3 999
999 8 9 4 999
999 7 6 5 999
999 999 999 999 999

```

Tabel 5 Hasil Uji tc4.txt

Algoritma	Execution Time (ms)	Iterations, Path Length	Total cost	Screenshot
A*	1	3 iterations, 1 steps	13	 Path Found: 0-1-2 Algorithm (A*) Execution Time: 1 ms Iterations: 3 iterations Step: 1 / 1 Path Length: 1 steps Total Cost: 13
UCS	0	3 iterations, 1 steps	13	 Path Found: 0-1-2 Algorithm (UCS) Execution Time: 0 ms Iterations: 3 iterations Step: 1 / 1 Path Length: 1 steps Total Cost: 13

GBFS	0	2 iterations, 1 steps	13	
------	---	--------------------------	----	--

Pada pengujian ini, ketiga algoritma berhasil menemukan total *cost* yang identik karena jalur terpendek secara heuristik juga merupakan satu-satunya rute yang tersedia untuk mencapai target. GBFS terlihat lebih efisien dalam jumlah iterasi karena langsung menuju target tanpa melakukan validasi *cost* tambahan. Sementara itu, A* dan UCS membutuhkan iterasi ekstra untuk memastikan tidak ada jalur alternatif yang lebih murah. Kesamaan hasil *cost* menunjukkan bahwa pada peta dengan koridor sempit dan pilihan gerak terbatas, perbedaan strategi pencarian antar algoritma hanya berpengaruh pada kecepatan proses, bukan kualitas solusi akhir.

File tc4.txt

tc_soal.txt
<pre> 4 8 XXXXXXXXX XZ*0*10X X*****X XXXXXXXXX 999 999 999 999 999 999 999 999 999 1 1 9 1 1 1 999 999 2 3 2 4 2 5 999 999 999 999 999 999 999 999 999 </pre>

LAMPIRAN

Tabel Checklist

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	☒	
2	Program berhasil dijalankan	☒	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	☒	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	☒	
5	Program memiliki Graphical User Interface (GUI)	☐	
6	Ada algoritma <i>pathfinding</i> alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y)		☒
7	Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)		☒

Pranala

Tautan Github
https://github.com/ielita/Tucil3_18223101.git
Google Drive
 Deliverable Tucil 3

Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

A handwritten signature in black ink, appearing to read 'Sekar--', located in the bottom right corner of the statement box.

Ni Made Sekar Jelita Parameswari